

目录

一、项目概述.....	1
二、系统架构设计.....	2
2.1 架构模式说明	2
2.2 微服务架构图	3
三、项目功能模块分析.....	4
3.1 后台管理系统主要功能	4
3.2 前台用户系统主要功能	4
四、核心功能实现分析.....	6
4.1 用户注册与登录	6
4.2 购物车模块	10
五、系统部署与运行.....	13
六、总结与反思.....	25
6.1 整体感受与收获:	25
6.2 对 Spring Cloud 微服务架构的理解:	25
6.3 最具挑战的模块及解决方案	25
6.4 项目商业应用潜力分析	26
6.5 项目优化建议	26
6.6 老师提问的问题	27
附录.....	28

一、项目概述

SPZX 商城是一个基于 Spring Cloud 微服务架构的电商平台，采用前后端分离的开发模式。项目旨在构建一个功能完善、性能稳定的 B2C 电商系统，为用户提供商品浏览、购物车、订单管理、在线支付等一站式购物体验。

业务背景：随着电商行业的快速发展，传统单体架构已难以满足高并发、高可用的业务需求。SPZX 商城采用微服务架构，将系统拆分为多个独立服务，实现业务解耦和弹性伸缩。

核心功能模块：

- 用户模块：注册、登录、个人中心、地址管理
- 商品模块：商品展示、分类查询、商品详情
- 购物车模块：添加商品、数量修改、选中状态管理
- 订单模块：订单生成、订单列表、订单状态跟踪
- 支付模块：支付宝支付集成、支付状态回调
- 后台管理：商品管理、订单管理、用户管理、权限控制

系统架构设计：采用 Spring Cloud Alibaba 技术栈，结合 Nacos 注册中心、Gateway 网关、OpenFeign 服务调用等组件，构建高可用的微服务体系。前端采用 Vue3 + Vite 构建，通过 RESTful API 与后端通信。

微服务架构优势：

- 服务独立部署，互不影响
- 支持弹性伸缩，应对高并发
- 团队分工明确，提高开发效率
- 技术选型灵活，各服务可采用不同技术栈

前后端分离优势：

- 前端专注用户体验，后端专注业务逻辑
- 接口标准化，便于联调测试
- 支持多端适配（H5、小程序、APP）

二、系统架构设计

2.1 架构模式说明

后台系统采用单体架构：

- spzx-manager 作为后台管理服务，采用单体架构设计
- 包含商品管理、订单管理、用户管理、权限管理等完整功能
- 直接连接数据库和缓存，不经过 Gateway 网关
- 便于后台管理功能的快速开发和维护

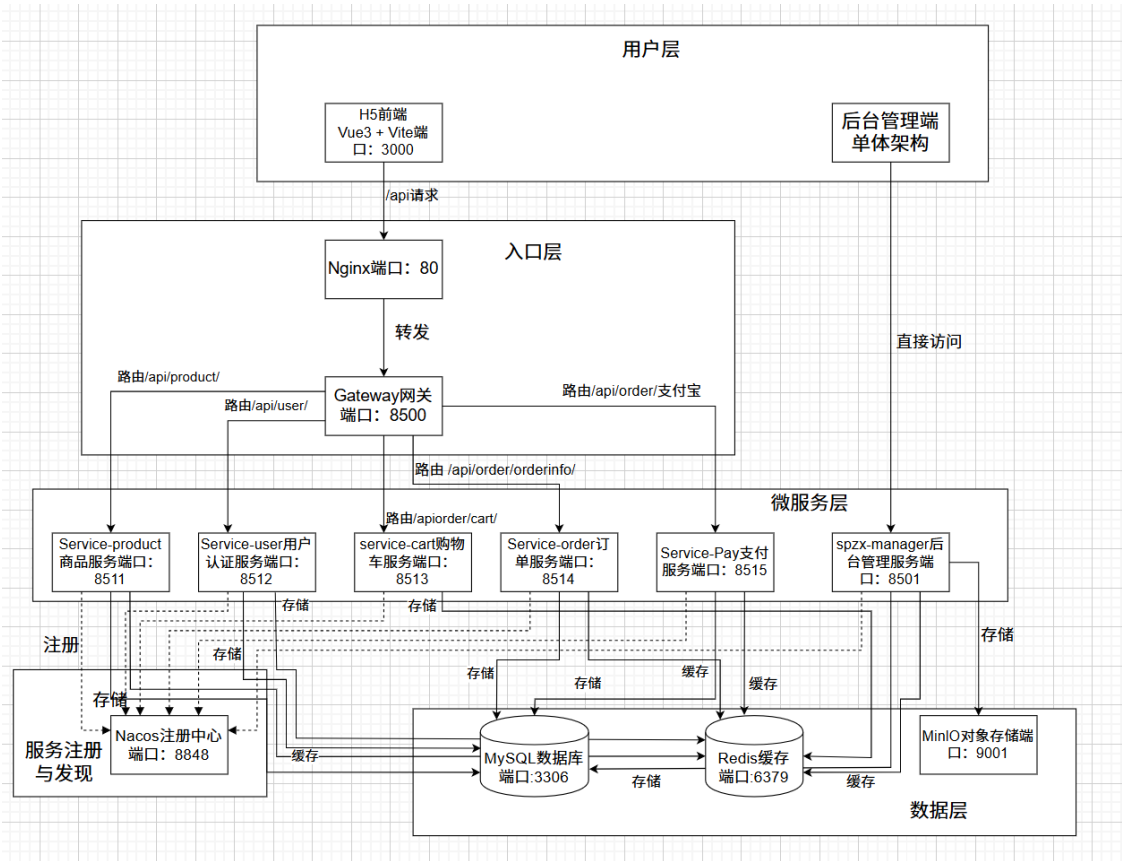
前台用户系统采用微服务架构：

- 基于 Spring Cloud Alibaba 构建，采用微服务拆分策略
- 各服务独立部署、独立运行，职责单一
- 通过 Nacos 注册中心实现服务发现和负载均衡
- 通过 Gateway 网关实现统一入口和路由转发

服务间通过接口协作，使用 JSON 数据格式：

- 前端通过 RESTful API 与后端通信
- 微服务间通过 OpenFeign 客户端进行调用
- 数据交互统一使用 JSON 格式
- 统一的响应结果封装（Result 对象）

2.2 微服务架构图



组件关系说明:

组件	端口	职责说明
Nacos	8848	服务注册与发现中心， 管理所有微服务的注册 信息
Gateway	8500	API 网关，统一入口， 负责路由转发和跨域处 理
service-product	8511	商品服务，处理商品相 关业务逻辑
service-user	8512	用户服务，处理用户注 册、登录、认证等业务
service-cart	8513	购物车服务，基于 Redis 实现购物车功能
service-order	8514	订单服务，处理订单创 建、查询等业务
service-pay	8515	支付服务，集成支付宝 支付功能

三、项目功能模块分析

3.1 后台管理系统主要功能

用户登录与权限管理（基于 Spring Security）：

- 管理员登录认证
- 基于角色的权限控制（RBAC）
- 菜单权限动态展示
- 用户管理、角色管理、菜单管理

商品管理：

- 商品信息维护（新增、修改、删除）
- 商品分类管理
- 商品规格管理
- 商品 SKU 管理
- 商品上下架操作

订单管理：

- 订单列表查询
- 订单状态跟踪
- 订单发货处理
- 订单统计分析

文件上传：

- 商品图片上传
- 商品详情图上传
- 基于 MinIO 对象存储
- 文件访问路径管理

3.2 前台用户系统主要功能

首页信息展示：

- 轮播图展示

- 分类导航
- 热门商品推荐
- 活动促销信息

商品浏览与分类查询:

- 商品列表分页展示
- 商品分类树查询
- 商品筛选（价格、销量等）
- 商品搜索功能

用户注册、登录流程:

- 手机验证码注册
- 账号密码登录
- Token 令牌认证
- 用户信息获取

购物车流程:

- 添加商品到购物车
- 修改商品数量
- 选中/取消选中商品
- 清空购物车

下单流程:

- 确认订单信息
- 选择收货地址
- 使用优惠券
- 提交订单

支付流程:

- 支付宝在线支付

- 支付状态查询
- 支付结果回调

四、核心功能实现分析

4.1 用户注册与登录

1.功能目标描述:

实现用户注册和登录功能，支持手机号验证码注册和账号密码登录，登录成功后返回 Token 令牌用于后续接口认证。

2.前端与后端实现逻辑简述

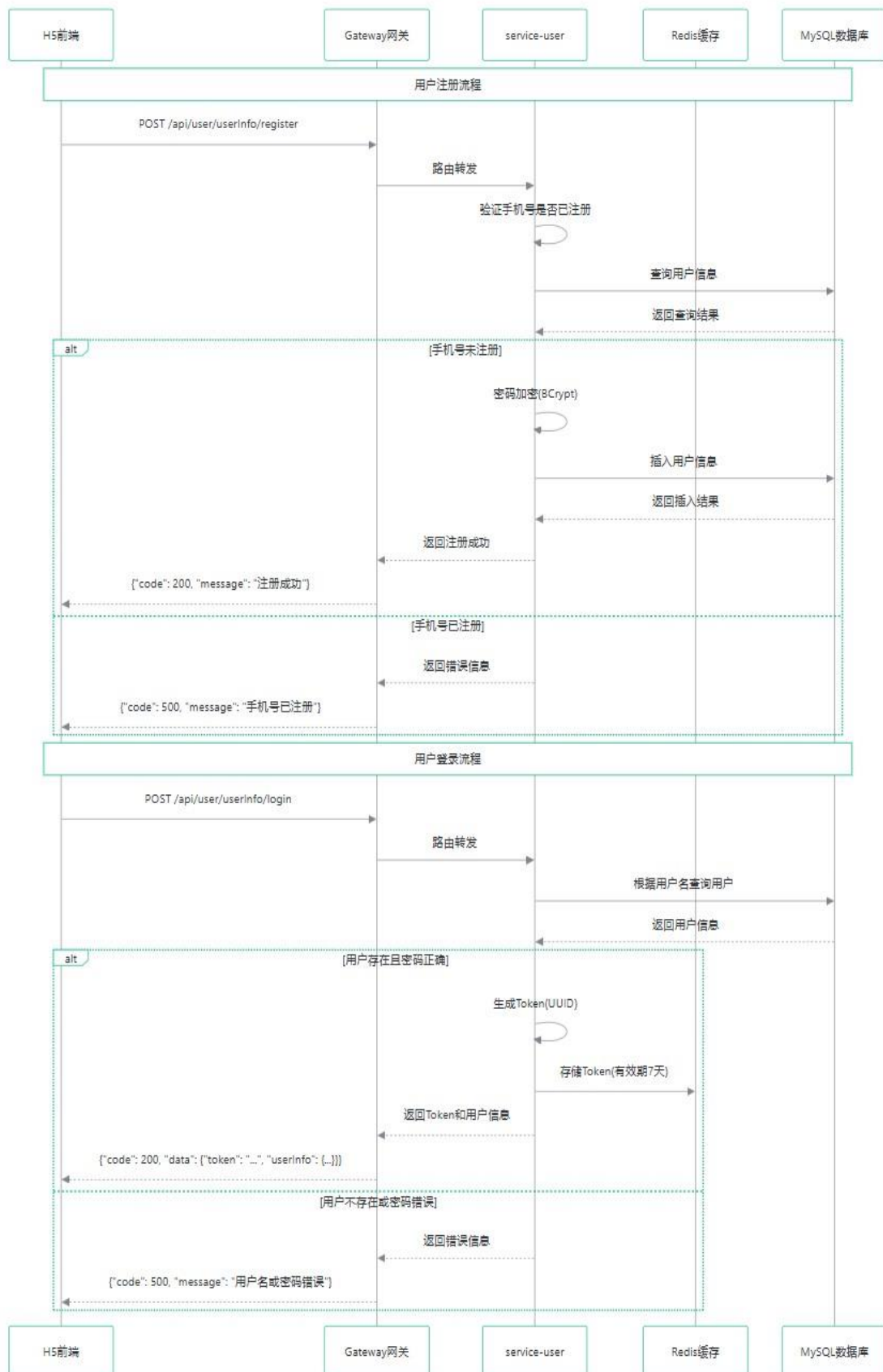
前端实现:

- 注册页面：收集用户名、密码、手机号、验证码
- 登录页面：收集用户名、密码
- 使用 axios 发送请求
- 登录成功后将 Token 存储在 localStorage
- 请求拦截器自动携带 Token

后端实现:

- UserInfoController 接收注册和登录请求
- UserInfoService 处理业务逻辑
- 注册：验证手机号、密码加密、保存用户信息
- 登录：验证用户名密码、生成 Token、存储 Token 到 Redis

3.核心流程图:



4.主要代码逻辑片段

前端 API 调用（user.js）：

```
export const login = (data) => request.post('/user/userInfo/login', data)
export const register = (data) => request.post('/user/userInfo/register', data)
export const getUserInfo = () => request.get('/user/userInfo/auth/getCurrentUserInfo')
```

后端 Controller（UserInfoController.java）：

```
@Operation(summary = "会员注册")
```

```
@PostMapping("register")
```

```
public Result register(@RequestBody UserRegisterDto userRegisterDto) {
    userInfoService.register(userRegisterDto);
    return Result.build(null, ResultCodeEnum.SUCCESS);
}
```

```
@Operation(summary = "会员登录")
```

```
@PostMapping("login")
```

```
public Result login(@RequestBody UserLoginDto userLoginDto) {
    return Result.build(userInfoService.login(userLoginDto),
        ResultCodeEnum.SUCCESS);
}
```

```
@Operation(summary = "获取当前登录用户信息")
```

```
@GetMapping("auth/getCurrentUserInfo")
```

```
public Result<UserInfoVo> getCurrentUserInfo(HttpServletRequest request) {
    String token = request.getHeader("token");
    UserInfoVo userInfoVo = userInfoService.getCurrentUserInfo(token);
    return Result.build(userInfoVo, ResultCodeEnum.SUCCESS);
}
```

5.遇到的技术问题与解决方法

问题 1：跨域问题

- 现象：前端请求后端接口时出现 CORS 跨域错误
- 解决方法：在 Gateway 网关配置全局 CORS 跨域规则

spring:

cloud:

gateway:

globalcors:

cors-configurations:

'[/**]':

allowedOriginPatterns: "*"

allowedHeaders: "*"

allowedMethods: "*"

maxAge: 36000

问题 2：Token 过期处理

- 现象：Token 过期后接口返回 401 错误
- 解决方法：前端请求拦截器统一处理 401 状态码，跳转登录页面

```
request.interceptors.response.use(response => {  
    // 成功处理  
}, error => {  
    if (error.response && error.response.status === 401) {  
        showToast('请先登录')  
        // 跳转登录页  
    }  
    return Promise.reject(error)  
})
```

4.2 购物车模块

1.功能目标描述:

实现购物车功能，支持添加商品、修改数量、选中状态管理、清空购物车等操作，基于 Redis 实现高效读写。

2.前端与后端实现逻辑简述

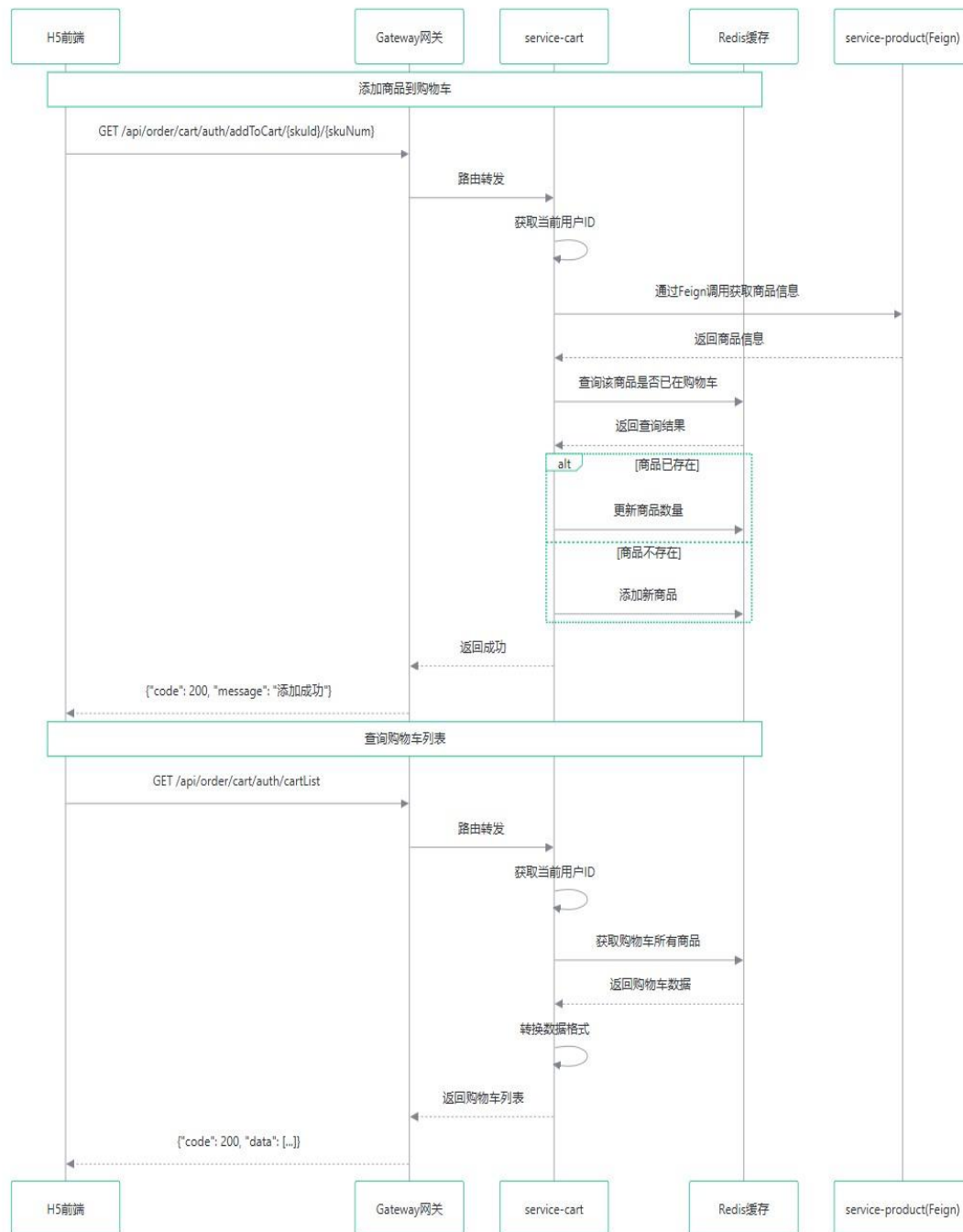
前端实现:

- 购物车页面展示商品列表
- 支持商品数量增减
- 支持单选/全选商品
- 实时计算选中商品总价
- 提交选中商品生成订单

后端实现:

- CartController 提供购物车相关接口
- CartService 处理购物车业务逻辑
- 使用 Redis Hash 存储购物车数据 (key: cart:userId, field: skuId, value: cartInfoJson)
- 用户未登录时使用临时 key，登录后合并购物车

3.核心流程图:



4.主要代码逻辑片段

前端 API 调用（cart.js）：

```

export const addToCart = (skuId, skuNum) =>
request.get(`/order/cart/auth/addToCart/${skuId}/${skuNum}`)

export const getCartList = () => request.get('/order/cart/auth/cartList')
    
```

```

export const deleteCart = (skuId) =>
request.delete(`/order/cart/auth/deleteCart/${skuId}`)

export const checkCart = (skuId, isChecked) =>
request.get(`/order/cart/auth/checkCart/${skuId}/${isChecked}`)

export const allCheckCart = (isChecked) =>
request.get(`/order/cart/auth/allCheckCart/${isChecked}`)

export const clearCart = () => request.get('/order/cart/auth/clearCart')

```

后端 Controller（CartController.java）：

```

@Operation(summary = "添加购物车")

@GetMapping("auth/addToCart/{skuId}/{skuNum}")

public Result addToCart(@PathVariable("skuId") Long skuId,

                        @PathVariable("skuNum") Integer skuNum) {

    cartService.addToCart(skuId, skuNum);

    return Result.build(null, ResultCodeEnum.SUCCESS);

}

@Operation(summary = "查询购物车")

@GetMapping("auth/cartList")

public Result<List<CartInfo>> cartList() {

    List<CartInfo> cartInfoList = cartService.getCartList();

    return Result.build(cartInfoList, ResultCodeEnum.SUCCESS);

}

@Operation(summary = "更新购物车商品选中状态")

@GetMapping("auth/checkCart/{skuId}/{isChecked}")

public Result checkCart(@PathVariable(value = "skuId") Long skuId,

                        @PathVariable(value = "isChecked") Integer isChecked) {

    cartService.checkCart(skuId, isChecked);

```

```
return Result.build(null, ResultCodeEnum.SUCCESS);  
}
```

5.遇到的技术问题与解决方法

问题 1：用户未登录时购物车数据存储

- 现象：用户未登录时无法将购物车数据关联到用户
- 解决方法：使用浏览器本地存储（localStorage）暂存，登录后同步到 Redis

问题 2：购物车数据一致性

- 现象：商品价格或库存发生变化时，购物车数据未同步更新
- 解决方法：查询购物车时，调用商品服务获取最新价格和库存信息，实时更新购物车数据

问题 3：购物车缓存过期

- 现象：Redis 缓存过期导致购物车数据丢失
- 解决方法：设置合理的缓存过期时间（如 7 天），并在用户访问购物车时自动续期

五、系统部署与运行

1. 项目部署流程

环境准备

- JDK 17+
- Maven 3.8+
- MySQL 8.0+
- Redis 7.0+
- Nacos 2.2.2
- Node.js 18+

本地运行步骤

步骤 1：启动基础服务

启动 Nacos

```
cd E:\nacos\bin
```

```
startup.cmd -m standalone
```

启动 Redis

```
redis-server redis.conf
```

启动 MySQL（确保数据库已创建）

```
mysql -u root -p
```

```
CREATE DATABASE db_spzx CHARACTER SET utf8mb4;
```

步骤 2：构建后端项目

```
cd E:\qiye\spzx-parent(5)\spzx-parent
```

```
mvn clean install -DskipTests
```

步骤 3：启动微服务（按顺序启动）

(1) spzx-server-gateway（端口 8500）

(2) service-product（端口 8511）

(3) service-user（端口 8512）

(4) service-cart（端口 8513）

(5) service-order（端口 8514）

(6) service-pay（端口 8515）

(7) spzx-manager（端口 8501）

步骤 4：启动前端项目

```
cd E:\qiye\spzx-parent(5)\spzx-h5-vue3(3)\spzx-h5-vue3
```

```
npm install
```

```
npm run dev
```

2. Nacos 注册中心、Gateway 网关配置说明

Nacos 配置

所有微服务通过以下配置注册到 Nacos:

spring:

cloud:

nacos:

discovery:

server-addr: 192.168.198.142:8848

Gateway 网关路由配置

spring:

cloud:

gateway:

routes:

- id: service-product

uri: lb://service-product

predicates:

- Path=/*/product/**

- id: service-user

uri: lb://service-user

predicates:

- Path=/*/user/**

- id: service-cart

uri: lb://service-cart

predicates:

- Path=/api/order/cart/**

- id: service-order

uri: lb://service-order

predicates:

- Path=/api/order/orderInfo/**

```
- id: service-pay  
uri: lb://service-pay  
predicates:  
  - Path=/api/order/alipay/**
```

3. 微服务间调用方式

OpenFeign 调用

微服务间通过 OpenFeign 客户端进行调用，例如订单服务调用商品服务：

```
@FeignClient(value = "service-product")
```

```
public interface ProductFeignClient {
```

```
    @GetMapping("/api/product/getBySkuId/{skuId}")
```

```
    ProductSku getBySkuId(@PathVariable Long skuId);
```

```
    @PostMapping("/api/product/updateSkuSaleNum")
```

```
    Boolean updateSkuSaleNum(@RequestBody List<SkuSaleDto> skuSaleDtoList);
```

```
}
```

4. 项目调试与接口测试工具

Knife4j 接口文档

项目集成了 Knife4j，启动服务后访问以下地址查看接口文档：

- Gateway 网关：http://localhost:8500/doc.html
- 各微服务：http://localhost:端口/doc.html

接口测试示例

用户注册

POST http://localhost:8500/api/user/userInfo/register

Content-Type: application/json

```
{  
  "username": "testuser",  
  "password": "123456",  
  "phone": "13800138000",  
  "code": "1234"  
}
```

用户登录

POST http://localhost:8500/api/user/userInfo/login

Content-Type: application/json

```
{  
  "username": "testuser",  
  "password": "123456"  
}
```

获取商品列表

GET http://localhost:8500/api/product/1/10

添加购物车

GET http://localhost:8500/api/order/cart/auth/addToCart/1/1

Header: token=xxx

5. 系统运行截图说明

Nacos 服务注册列表:

- 访问地址: http://localhost:8848/nacos
- 可看到所有微服务实例已注册成功